CSCI 4100
Operating Systems and Architecture

Chapter 7: Scheduling

Topics

- 7.0 – Scheduling
- 7.1 – Uniprocessor Scheduling
- 7.2 – Multiprocessor Scheduling
- 7.3 – Energy-Aware Scheduling
- 7.4 – Real-Time Scheduling
- 7.5 – Queueing Theory
- 7.6 – Overload Management
- 7.7 – Case Study: Servers in a Data Center

Scheduling

- A **processor scheduling policy** determines which threads to run first when there are more threads than processors.
- A **task** or a **job** is a user request that is the focus of scheduling policy.
- **Response time** is the user-perceived time to perform a task.
- **Predictability** is a low variance in response times for repeated requests.
- **Throughput** is the rate at which tasks are completed.
- **Scheduling overhead** is the time it takes to switch from one task to another.
- **Fairness** refers to equal resource allocation for each task.
- **Starvation** occurs when a task fails to make progress due to resources given to a higher priority task.

Uniprocessor Scheduling

- A **workload** is a set of tasks for some system to perform.
- A **compute-bound** task spends most of its time using the processor.
- An **I/O-bound** task spends most of its time waiting for I/O operations to complete.
- A scheduler is **work-conserving** if it never leaves the processor idle when tasks are available.
- Most schedulers have the ability to **preempt** the processor and give it to another task.

First In First Out

The simplest scheduling policy is **First In First Out (FIFO)** or **First Come First Served (FCFS)**.

- Tasks are performed in the order in which they arrive, and keep running until they complete.
- Advantages: simple to implement and minimizes overhead.
- Disadvantages: shorter jobs may have to wait for longer jobs to complete, increasing average response time.

Shortest Job First

The optimal policy for average response time is **Shortest Job First (SJF)** or **Shortest Remaining Time First (SRTF).**

- Always schedule the task that has the least remaining work to do first.
- Advantages: minimizes average response time.
- Disadvantages: requires knowledge of the future, longer tasks have poor response time and may starve.

Round Robin

To prevent starvation, systems can use a **Round Robin** policy.

- Each task takes turns running on the processor for a limited period of time called a **time quantum** or **time slice**.
- Advantages: every task will eventually get to run, preventing starvation.
- Disadvantages:
  - A short time quantum may add overhead and increase average response time for some workloads.
  - A long time quantum will behave just like FIFO for other workloads, also increasing average response time.
  - I/O-bound tasks are more sensitive to delay than compute-bound tasks, but Round Robin treats them the same.

Max-Min Fairness

To make sure each process gets its fair share of the processor, we can enforce a **max-min fairness** policy.

- If all processes are compute-bound, each one gets an equal share of the processor.
- If a process does not use its entire share, the remainder is distributed among the remaining processes.
- We can implement this by always giving the processor to the process that has used the processor the least.
- To avoid frequent switching, a process may be allowed to get one quantum ahead of its fair allocation.
- Advantages: maximizes fairness.
- Disadvantages: keeping track of allocation shares may impose significant overhead.

Multi-Level Feedback Queue

Most operating systems use a **multi-level feedback queue (MFQ)** to balance various scheduling goals.

- Multiple queues are used to keep track of tasks, each with a different priority level.
- Higher priority tasks preempt lower priority tasks, but run with a shorter time quantum.
- New tasks are put on the highest priority queue.
- If a task uses up its entire quantum it drops a level.
- If a task yields the processor to wait on I/O it stays at the same level or moves up.
- To prevent starvation and improve fairness, processes that have not received their fair share can be increased in priority.

Multiprocessor Scheduling

A centralized MFQ with a lock may lead to performance problems.

- Contention for the lock may become a bottleneck.
- The state of the MFQ will need to be fetched from the cache of the last processor to access it, adding overhead.
- Threads may be scheduled on processors that do not have their data cached, impacting performance.

Affinity Scheduling

Most systems give each processor its own MFQ.

- Each processor uses **affinity scheduling**, where threads are scheduled on the processor they last ran on.
- If the MFQs become too imbalanced, threads can be moved to different MFQ to improve processor utilization.
- The MFQs must still be protected by locks, but the overhead of lock contention and cache coherence is must less than the single-MFQ approach.

Oblivious Scheduling

An **oblivious scheduling** approach treats parallel threads as if they were independent of one another.

- Applications using **bulk synchronous parallelism**, where each thread blocks until all threads complete a stage of computation, may be stalled if one thread is preempted.
- Applications using a producer-consumer approach can be stalled if one thread in a chain is preempted.
- In general, preempting threads in the **critical path** of a parallel application can slow down progress.
- If the holder of a lock is preempted, threads waiting to acquire the lock will not be able to make progress.

Gang Scheduling

To address these issues, a system can use **gang scheduling**.

- All threads of an application must run together or not at all.
- Most systems allow a set of processors to be dedicated to a single application.
- Some applications may see diminishing returns when adding additional processors.
- The number of processors available may change over time.

Scheduler Activations

An alternative approach is to use **scheduler activations**.

- An application is informed by the OS of each processor assigned to it via an upcall.
- If a thread blocks, the processor it is running on is reassigned to another application.
- The OS schedules applications on the processors, and an application schedules its threads on the scheduler activations.
- A multiprocessor scheduling policy is still required to determine how many processors to assign each application.

Energy-Aware Scheduling

Scheduling decisions can also impact energy usage.

- Low power processors are usually slower and simpler.
- Some systems have a high-power processor for performance and a low power processor to conserve energy.
- On multicore systems lightly used processors can be disabled to save power.
- I/O devices not in use can be powered off.
- Optimizing for power consumption affects performance, so it is important to consider the impact on response time when making these decisions.

Real-Time Scheduling

Some systems have **real-time constraints** that require computations to be completed by a deadline.

- Real-time tasks can be allocated more processing power than they need to ensure they complete quickly.
- An **earliest deadline first (EDF)** policy can be used to satisfy real-time constraints
- If all tasks are CPU-bound and can complete before their deadline, EDF guarantees the deadlines are met.
- I/O-bound tasks may need to be broken down into subtasks, each with its own deadline.

Shared data structures can impact real-time scheduling.

- **Priority inversion** occurs when a high priority task must wait for a lower priority task to complete its work.
- Most systems allow **priority donation**, where a high priority task can donate its priority to a lower priority task it is waiting for.

Queuing Theory

Queuing theory is concerned with the performance of a system that queues requests before servicing them.

- A **server** is anything that performs tasks.
- We shall assume that a server handles one task at a time
- To measure the response time R for a task we use the formula $R = W + S$, where W is the **waiting time** and S is the **service time**.
- To measure the utilization U of a server we use the formula $U = \min(\lambda/\mu\,, 1)$, where λ is the **arrival rate** for new tasks and μ is the **service rate** for processing tasks.
- To measure the throughput X of a server we use the formula $X = \min(\lambda, \mu)$.
- To measure the number of tasks N in the system we use the formula $N = Q + U$, where Q is the number of tasks queued.

Little's Law

Little's Law states that in a **stable system** where the arrival rate matches the departure rate, that $N = XR$.

- Consider a system that processes 100 requests per second with a response time of 50ms.
- The system will have $N = XR = 100 \cdot 0.05 = 5$ requests in the queue or being serviced on average.
- Now assume that the response time R consists of a waiting time W of 45ms and a service time S of 5ms.
- The average utilization is $U = XS = 100 \cdot 0.005 = 0.5$ or 50%.
- The average number of requests queued is $Q = XW = 100 \cdot 0.045 = 4.5$.

Response Time Versus Utilization

Maximizing the utilization of a system will typically improve the throughput as long as the arrival rate doesn't exceed the service rate.

- Higher utilization can also impact the response time.
- In the best case, arrivals are evenly spaced so the response time is constant as long as $\lambda < \mu$.
- In the worst case, requests arrive in bursts so the response time increases with the arrival rate.
- In either case, once $\lambda > \mu$ the system becomes overloaded and the response time grows without bound.

Exponential Arrivals

In most systems requests arrive randomly, and can be modeled using an **exponential distribution**.

- An exponential distribution is **memoryless**, which means that recent history can not be used to predict the future.
- If we assume that $\lambda < \mu$ and an exponential distribution for the arrival and departure of requests, then $R = \frac{S}{1-U}$.
- In a system that is 20% utilized, the response time is $\frac{S}{1-0.2} = 1.25S$.
- If the utilization goes up by 5%, the response time is $\frac{S}{1-0.25} = 1.33S$.
- If the system is 90% utilized and utilization goes up by 5%, the response time goes from $10S$ to $20S$.

Workload and Scheduling Policy

The distribution of the workload impacts the performance of scheduling policies.

- In an evenly spaced workload FIFO performs better than Round Robin.
- In an exponentially distributed workload FIFO and Round Robin will have the same performance.
- In a bursty workload Round Robin performs better than FIFO.
- SJF performs well if there is a lot of variance in task lengths, but long tasks will have high response times when utilization is high.

Other Factors Impacting Performance

- If the arrival of new requests depends on the completion of previous requests, the arrival rate may go down when the queue fills up.
- A system with multiple servers could use a single queue or a queue for each server.
  - A queue for each server is more efficient in cases like multiprocessor scheduling.
  - A single queue generally provides lower average response time since requests can be serviced by the least busy server.
- Even if additional resources are added to improve average response time, there may be additional bottlenecks (like I/O) that arise.

Lessons

- Almost all real-world systems exhibit some randomness in arrival rate and/or service times.
- Response time increases with increased load.
- System performance is predictable if average service time can be estimated.
- Burstiness increases average response time.
- Exponential distribution is a useful model, but many real-world systems exhibit more burstiness.

Overload Management

- If more requests arrive than a system can handle, the system is **overloaded**.
- To provide good service to some requests, other requests may need to be rejected.
  - If a streaming video service is overloaded it can limit the number of viewers to maintain the quality of the video.
- Another approach is to reduce the service time by replacing the service with a faster one.
  - A web server could serve static web pages instead of dynamically generating them.
- Many systems will take the opposite approach and end up doing more work when a system is overloaded, making the situation worse.

Case Study: Servers in a Data Center

- Many web services provide front-end machines to direct requests to back-end machines that service them.
- When a client first connects, the front-end node assigns it to a back-end server to balance load.
- Additional requests from the same client can be directed to the same back-end server to improve response time.
- Priority scheduling can be used to prevent users from hogging resources.
- To minimize response time, back-end servers can be added when average utilization gets too high.
- Bringing new servers online takes time, so plans should be made to deal with overload conditions in the meantime.